Exploring Width, Depth, and Generalization in MLPs: The Quick, Draw! Challenge

Abdul Hafeez [1]

Abstract

This report investigates how width and depth trade-offs in Multi-Layer Perceptrons (MLPs) affect classification performance on the Quick, Draw! dataset, a collection of 28×28 grayscale doodles spanning 15 classes, under strict constraints of at most 3 million parameters and 40 training epochs. We compare a shallow wide "Pancake" model and a deep narrow "Tower" model, both of which plateau near 76–77% validation accuracy due to overfitting on flattened pixel inputs. Our final "Champion" model addresses these limitations through a progressive funnel architecture paired with SELU activations, LeCun initialization, Alpha Dropout, and spatial data augmentation, achieving a test accuracy of **80.75%** with only 0.45M parameters and 20 epochs. Code is available at https://github.com/hafeez381/Deep-Learning/tree/main/assignment_2.

1. Introduction

The "Quick, Draw!" dataset (Jongejan et al., 2017) is a large-scale collection of hand-drawn sketches crowdsourced from players of an online game. For this assignment, we use a subset of the data consisting of 28×28 grayscale doodles. The primary objective is to accurately classify these images into 15 distinct categories using only flat vectors, subject to three constraints:

1. **Architecture:** MLPs only (no CNNs).
2. **Parameters:** Maximum 3 million trainable parameters.
3. **Epochs:** Maximum 40 training epochs.

1.1. Exploratory Data Analysis

Before designing the models, we conducted Exploratory Data Analysis (EDA) to better understand the dataset's characteristics.

1. Visualizing the Data

As illustrated in Figure 1, the doodles are highly abstract. Because they are drawn in a matter of seconds, there is immense intra-class variation, making classification challenging for MLPs that lack the spatial awareness of CNNs.

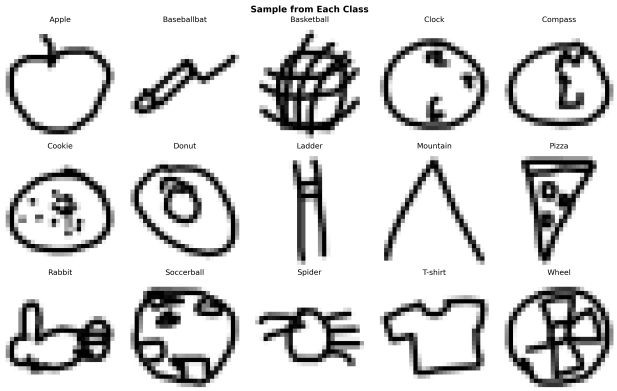

Figure 1. **Sample from Each Class.** A random 28×28 grayscale doodle from all 15 categories, highlighting the abstract and noisy nature of the dataset.

2. Class Distribution

We also analyzed the frequency of each category in the training set ($N = 60,000$ samples) to check for class imbalance. As shown in Figure 2, the dataset is perfectly balanced, with exactly 4,000 samples per class. This ensures that the model will not become biased toward a majority class during training.

1.2. Training Methodology and Pipeline

The training data was partitioned 80/20 into 48,000 training and 12,000 hold-out validation samples. All models minimized Categorical Cross-Entropy using AdamW (lr $= 0.001$, weight decay 10^{-4}). Early Stopping (patience $p = 5$) monitored the validation loss and restored the best checkpoint upon non-improvement.

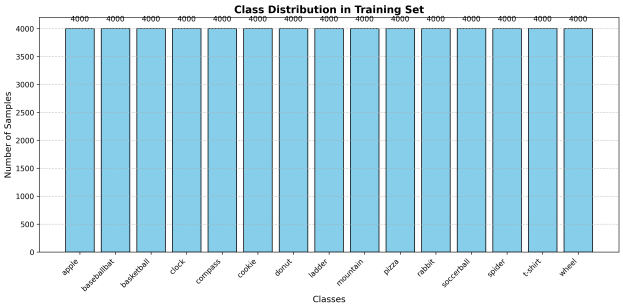

Figure 2. **Class Distribution in Training Set.** The training dataset contains an equal distribution of 4,000 samples for each of the 15 classes.

2. Architecture Experiments

2.1. Part A: The Pancake Model (Width Focus)

To test whether raw memorization power can solve this image classification task, we first built a shallow but extremely wide network, which we call the "Pancake" model.

2.1.1. Architecture and Capacity

The 28×28 image is flattened to a 784-dimensional vector and passed through a single hidden layer of 2,048 ReLU neurons. Dropout ($p = 0.3$) is applied before the final linear layer that outputs 15 class logits. The network totals approximately **1.64 million** trainable parameters—over half the budget in a single transformation.

2.1.2. Does Memorization Solve the Problem?

Based on our training logs, the Pancake model rapidly overfit the data. While the training accuracy climbed to **90.20%** by epoch 11, the validation accuracy peaked much earlier at epoch 6, reaching only **76.62%** (with a minimum validation loss of 0.7631). At this point, the early stopping mechanism successfully triggered, halting optimization to prevent further degradation. This severe generalization gap ($\approx 13.5\%$) indicates that even though extreme width gives the model enough capacity to memorize specific pixel arrangements, it struggles to generalize those abstract patterns to unseen doodles effectively.

While this immense raw memorization power allows the network to easily fit the training data, it is not the most efficient way to solve image classification problems. High parameter counts in a single wide layer fail to build the hierarchical feature representations (e.g., combining simple edges into complex shapes) that deep networks naturally learn. Consequently, the model is highly prone to memorizing noise.

2.2. Part B: The Tower Model (Depth Focus)

2.2.1. Architecture and Capacity

In contrast to the extreme width of the Pancake model, the "Tower" model explores the efficacy of depth. We constructed a highly narrow but deep Multi-Layer Perceptron consisting of six consecutive hidden layers, each containing only 256 neurons. The network flattens the 784-dimensional input and immediately compresses it to 256 dimensions, where it remains for the depth of the network before mapping to the 15 output classes.

This deep hierarchical structure drastically reduces the overall capacity of the network. The Tower model contains just $536,847$ trainable parameters—less than a third of the Pancake model's and well below our 3 million parameter budget.

2.2.2. Training Difficulties and Mitigation Strategies

Training deep, narrow MLPs introduces significant optimization challenges, most notably the *vanishing gradient problem*. As the error signal propagates backward through six sequential linear transformations and activation functions, it tends to decay exponentially. To combat this instability, we implemented **Batch Normalization** (`nn.BatchNorm1d`) immediately after every linear layer, prior to the ReLU activations.

Batch Normalization stabilizes the learning process by re-centering and re-scaling the pre-activations at each layer. This prevents the gradients from vanishing, allowing the optimization signal to reach the earliest layers of the deep network effectively.

2.2.3. Hierarchical Feature Learning vs. Parameter Count

The Tower model was trained with the same AdamW optimizer (lr $= 0.001$, weight_decay $= 10^{-4}$), but required an increased early stopping patience ($p = 8$) to navigate potential saddle points in the deeper loss landscape. The model achieved a peak validation accuracy of **76.95%** at epoch 9 (with a corresponding training accuracy of 87.57%). While the Tower model achieved a slightly higher validation accuracy than the Pancake model (76.95% vs 76.62%) while utilizing 67% fewer parameters, the absolute performance gain is marginal.

Deep networks usually excel at vision by learning in stages: early layers identify simple edges, while deeper layers combine them into complex objects. However, because an MLP requires flattening the 28×28 image into a single vector, it loses "spatial locality", the knowledge of which pixels are neighbors. Without this visual layout, a deep MLP cannot

build a meaningful hierarchy like a CNN. While depth is a more parameter-efficient way to map inputs to outputs, simply forcing a "deep but narrow" structure on flattened pixels does not guarantee better feature extraction.

2.3. Part C: The Champion Model

2.3.1. Architecture and Capacity

The final architecture is a 4-layer Multi-Layer Perceptron. The hidden layers progressively compress the 784-dimensional input feature space through dimensions of 400, 256, and 128 before outputting the 15 class logits. This structure forces the network to learn robust, low-dimensional representations of the doodles. The model contains exactly **451,487** trainable parameters, utilizing only about 15% of the maximum parameter budget.

2.3.2. Self-Normalizing Neural Network (SNN) Components

To ensure stable gradient flow without the computational overhead of Batch Normalization, we designed the Champion model as a Self-Normalizing Neural Network (Géron, 2025) with the following architectural choices:

Activation Function (`SELU`):
We replaced the standard ReLU with the Scaled Exponential Linear Unit (SELU). Unlike ReLU, SELU allows the network to self-normalize, meaning the outputs of each layer automatically tend to preserve a mean of 0 and a standard deviation of 1 during training.

Weight Initialization (`LeCun Normal`):
For SELU to properly self-normalize, the weights were initialized using LeCun Normal initialization ($\mu = 0, \sigma = \sqrt{1/\text{fan_in}}$), applied to all linear layers.

Regularization (`Alpha Dropout`):
Standard Dropout breaks the self-normalizing property of SELU by randomly zeroing out activations, shifting the mean. To combat overfitting while preserving self-normalization, we utilized Alpha Dropout (Géron, 2025), applying a 10% rate after the first hidden layer and a 5% rate after the second.

2.3.3. Data Augmentation and Preprocessing

Because MLPs lack the translational invariance inherent to CNNs, they are highly sensitive to where a doodle is drawn on the 28×28 canvas. To simulate real-world variations in human drawing strokes and improve generalization, we a spatial data augmentation pipeline using `torchvision.transforms`. During training, each doodle had a 50% chance of being horizontally flipped and was subjected to random affine transformations (up to 10 degrees of rotation and 8% spatial translation). Additionally, the pixel values were strictly standardized using the global mean and standard deviation of the training set to ensure the LeCun initialization behaved optimally.

2.3.4. Optimization and LR Scheduling

We optimized the network using **AdamW** with a base learning rate of 0.001 and a weight decay of 10^{-4}. To maximize convergence speed within our 20-epoch planned timeline, we implemented the **OneCycleLR** scheduler. The learning rate was warmed up to a maximum of 0.005 over the first 30% of training ($\approx$ 6 epochs) before undergoing cosine annealing. This aggressive scheduling helps the optimizer escape sharp local minima early on and settle into flat minima towards the end of training.

2.3.5. Results

The combination of heavy regularization (Alpha Dropout and Data Augmentation) successfully eliminated the generalization gap. Interestingly, throughout most of the training, the validation accuracy exceeded the training accuracy, probably due to the heavy training-time augmentation and dropout (which are disabled during validation). The Champion model achieved a peak **Validation Accuracy of 81.28%** at epoch 19, with a stable training accuracy of 78.95%. The **Test Accuracy**, after submission to the leaderboard, was found to be **80.75%**.

3. Model Comparison & Rationale

3.1. Performance Comparison Table

Table 1. Performance comparison of the explored MLP architectures.

Model	Params	Epochs	Train Acc	Val Acc
Pancake	1.64 M	11	87.50%	76.64%
Tower	0.54 M	13	87.57%	76.95%
Champion	0.45 M	20	78.95%	81.28%

3.2. Rationale for Architecture Selection

Depth and Width Trade-off: As shown in Table 1, both the Pancake and Tower networks rapidly memorized the training set ($\approx 87.5\%$ accuracy) while validation stagnated at $\approx$ 76.6–76.9%. Extreme width (Pancake, 1.64M params) wastes capacity on noise, while uniform depth (Tower, 256 neurons/layer) yields only marginal gains. For the Champion, we selected decreasing layer sizes (400 $\rightarrow$ 256 $\rightarrow$ 128). This "funnel" approach provides sufficient width in early layers to capture raw pixel combinations while using depth to progressively compress representations. This strat-

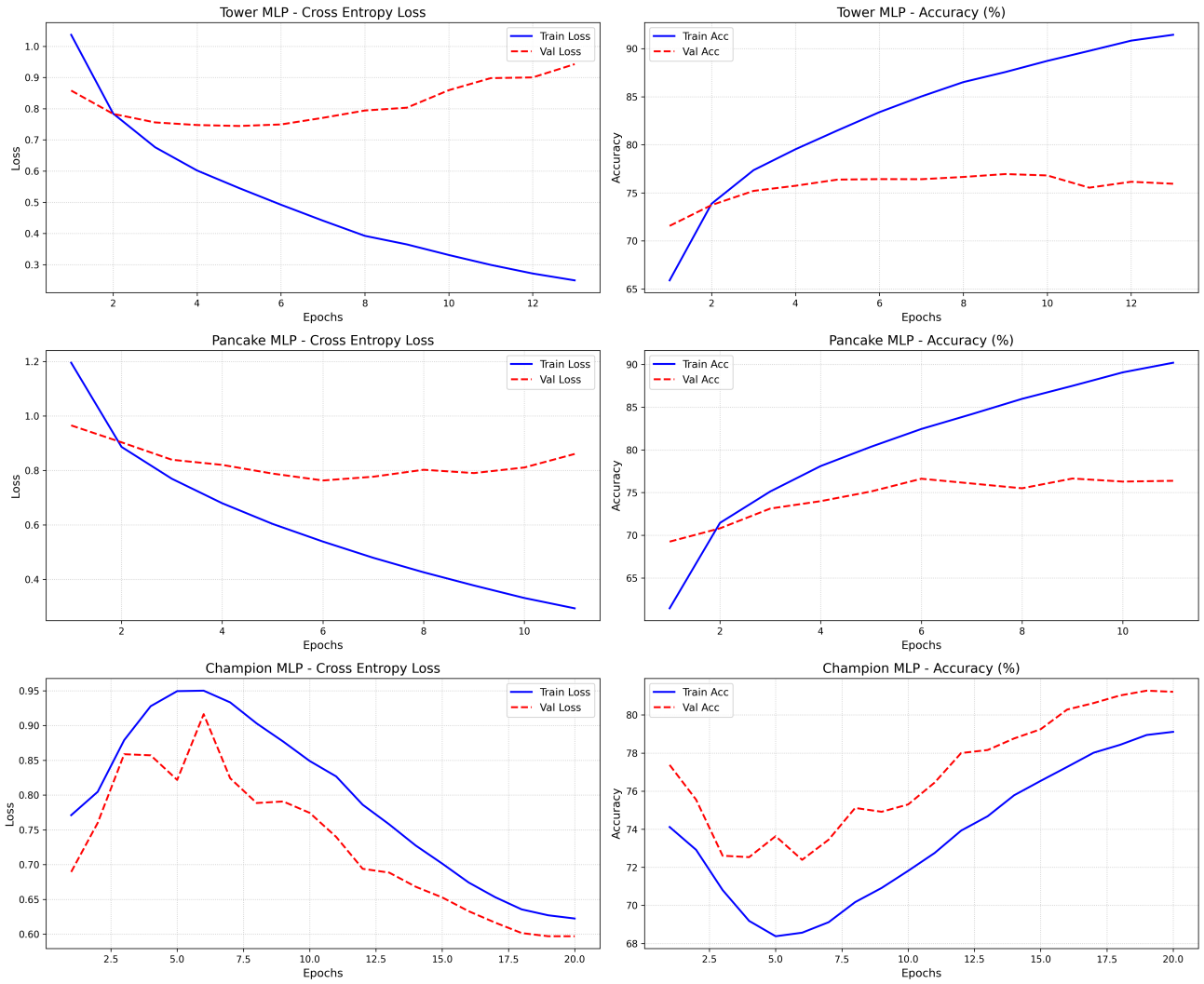

Figure 3. **Training and validation metrics.** The Pancake and Tower models display classic overfitting with diverging validation loss. In contrast, the Champion model demonstrates stable, regularized convergence without early halting.

egy achieved superior performance while maintaining the lowest parameter count (0.45M).

Combating Overfitting: The learning trajectories in Figure 3 show Pancake and Tower validation losses diverging sharply after epochs 5–6. The Champion employed spatial data augmentation and Alpha Dropout to close this gap; its validation accuracy consistently tracks *above* training accuracy, indicating it learned generalizable abstractions rather than memorized pixel layouts.

4. Theoretical Analysis

4.1. Width vs. Depth

Mathematically, the parameter count P for a single hidden layer MLP with input d, hidden units h, and classes k is $P = (d \times h + h) + (h \times k + k)$. For the wide Pancake model ($h = 2048$), this requires massive horizontal capacity, resulting in approximately 1.64×10^6 parameters. In contrast, the Tower model distributes its parameters across six stacked layers ($h = 256$), drastically reducing the mathematical footprint to 0.54×10^6 parameters.

According to the Universal Approximation Theorem, a single hidden layer with a finite number of neurons can theoretically approximate any continuous function to arbitrary

precision. Why, then, do we prefer deep architectures in practice?

While a shallow, wide network can map inputs to outputs, it essentially memorizes templates, requiring an exponentially large number of neurons to capture all possible variations. Deep architectures, however, break complex functions into a hierarchy of simpler sub-functions, extracting simple low-level edges in early layers and recursively combining them into complex, abstract shapes in deeper layers. This hierarchical reuse allows deep networks to generalize better and learn exponentially more complex mappings with. far fewer parameters

4.2. Confusion Analysis

To understand the Champion model's failure modes, we evaluated its predictions on the local validation set and generated a Confusion Matrix, presented in Figure 4.

Based on the off-diagonal densities, the top two most confused pairs of classes are:

1. **Pizza and Wheel:** The model predicted 'wheel' for a true 'pizza' 115 times, and predicted 'pizza' for a true 'wheel' 116 times.

2. **Compass and Clock:** The model predicted 'clock' for a true 'compass' 106 times, and predicted 'compass' for a true 'clock' 59 times.

Root Cause Analysis: Aleatoric vs. Epistemic Uncertainty

Visual inspection of the dataset, as shown in Figure 1, reveals that these confusions are heavily driven by **aleatoric uncertainty** (data ambiguity) rather than purely epistemic uncertainty (model limitations/underfitting).

The "Quick, Draw!" dataset consists of crude sketches drawn in under 20 seconds. A hastily drawn pizza (a circle with intersecting internal lines representing slices) is topologically identical to a hastily drawn wheel (a circle with intersecting internal lines representing spokes). Similarly, a clock and a compass are both basic circles containing internal arrows and perimeter tick marks.

Because the MLP flattens the image and lacks contextual priors it struggles to differentiate between classes that share the exact same geometric primitives. This suggests that the confusion stems largely from the noisy, ambiguous nature of the human sketches themselves rather than a lack of model capacity; even a human evaluator would struggle to separate these pairs reliably without labels.

References

Géron, A. *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Inc., 2025. ISBN 9798341607958.

Jongejan, J., Rowley, H., Kawashima, T., Kim, J., and Fox-Gieg, N. The Quick, Draw! Dataset. https://github.com/googlecreativelab/quickdraw-dataset, 2017.

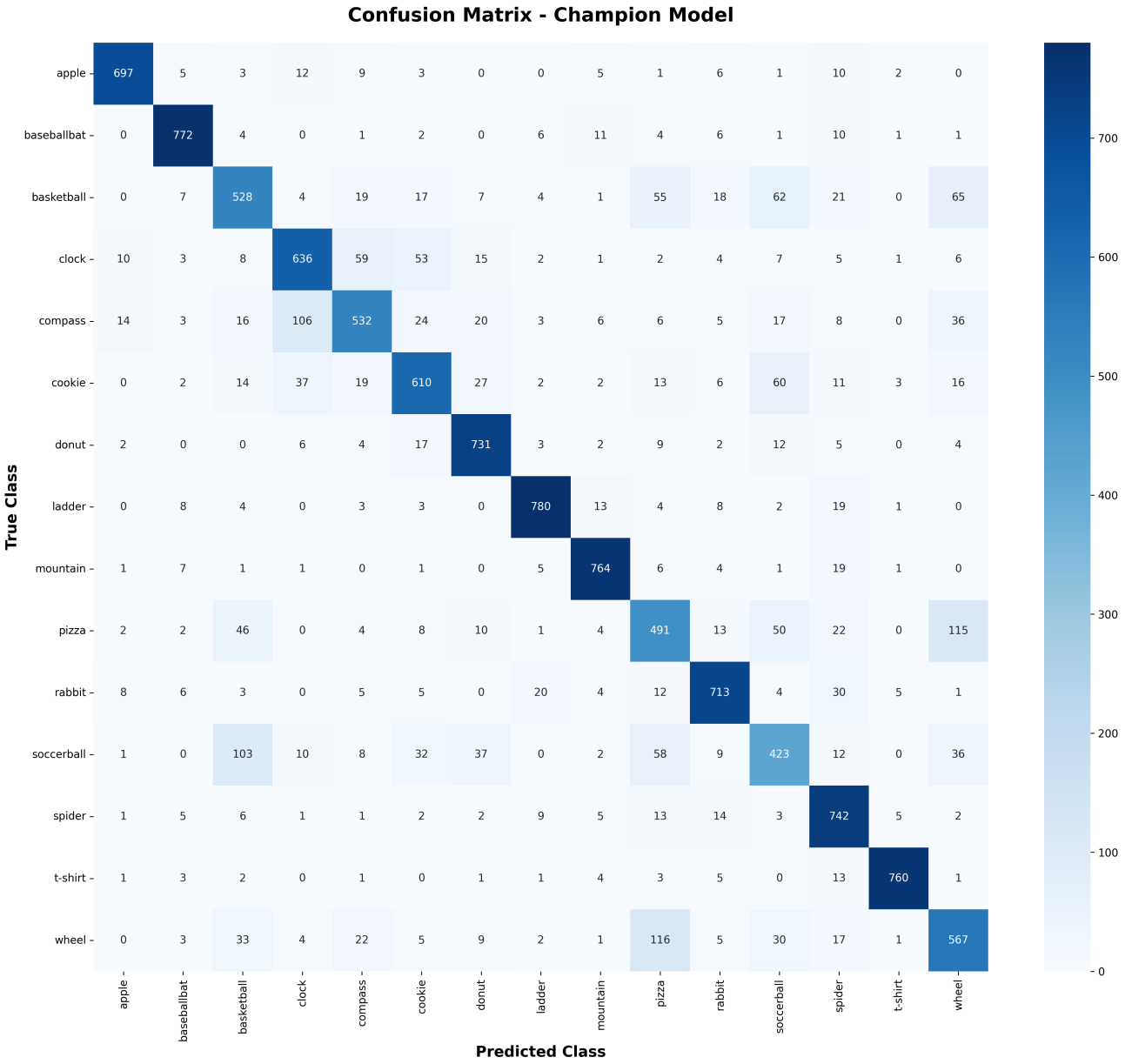

Figure 4. **Confusion Matrix for the Champion Model.** The diagonal represents correct classifications, while off-diagonal elements indicate confused classes.